

各相机投影模型

- u 和 v 表示图像原始坐标， x 和 y 表示主点中心的归一化图像坐标， XYZ 表示归一化笛卡尔相机坐标系坐标（不是方向向量，只是深度为1）
- 相机的成像可以理解为3D->笛卡尔相机坐标系->虚拟坐标系->归一化像平面->像素平面，不同模型关键在于虚拟坐标系不同，针孔该坐标系与相机坐标系相同，等距鱼眼模型成像的是角度信息只是在等距假设下这个角度和距离信息近似相等；球面展开则是成像的是经纬度；柱面展开成像的是一个固定柱面
- 由于相机畸变模型复杂，不具备解析解，强行迭代求解也容易大量空洞，所以通常只会用remap方式构造无畸变图像来重投影去畸变，而opencv的两个单点去畸变函数都是迭代近似求解，但因为针孔本身是多项式畸变，所以更容易迭代
- `cv::fisheye::undistortPoints(points_in, points_out, raw_k, raw_d, cv::noArray(), cv::noArray());`
这个鱼眼去畸变在迭代过程中使用的是 $r = f \tan \theta$ 假设，也就是变成了针孔的透视投影， $\tan \theta$ 在60°视场角以后的快速变化也就导致了这种投影方式的有效点非常有限

1. 针孔投影模型

1.1 虚拟内参

内参不需要调整，如果有缩放 f 和 c 需要乘上scale

1.2 像素投影相机

归一化相平面（深度为1平面）坐标为： $x = \frac{u - c_x}{f_x}, y = \frac{v - c_y}{f_y}$

相机坐标系下的点 $pt_cam = (d \cdot x, d \cdot y, d)$

或者另一种矩阵方法是 $K \cdot inv() \cdot (u, v, 1) / 1 \cdot d$

1.3 相机投影像素

齐次坐标转换 $x = X/Z, y = Y/Z$

加畸变

```

float r2 = x * x + y * y;
float coeff = 1 + raw_d.at<float>(0) * r2 + raw_d.at<float>(1) * r2 * r2 +
raw_d.at<float>(4) * r2 * r2 * r2;
if (raw_d.total() >= 8) {
coeff /= 1 + raw_d.at<float>(5) * r2 + raw_d.at<float>(6) * r2 * r2 +
raw_d.at<float>(7) * r2 * r2 * r2;
}
x2 = x * coeff + 2 * raw_d.at<float>(2) * x * y +
raw_d.at<float>(3) * (r2 + 2 * x * x);
y2 = y * coeff + raw_d.at<float>(2) * (r2 + 2 * y * y) +
2 * raw_d.at<float>(3) * x * y;

```

$$u = f_x * x' + c_x$$

$$v = f_y * y' + c_y$$

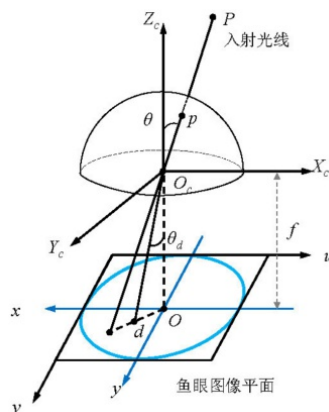
$$K(x',y',1)$$

2. 等距鱼眼模型

使用原点距离R，光轴夹角 θ 和光轴转角 ϕ 来描述相机坐标系上的点，并且将光轴夹角均匀投影在像平面上，使得相同夹角的点存在均匀投影，达到角度线性

$$r = f\theta$$

其中r为距离主点的像素长度 $r = \sqrt{(u - c_x)^2 + (v - c_y)^2}$,f为像素焦距，这样光轴夹角差值相等时对应了相同的像素变化，就导致中心区域信息密集， θ 较大时发生严重压缩，并且成像是一个圆（理想横纵f一致时）



2.1 虚拟内参

通常是一个360°的圆，所以图像是一个方形，向四周各辐射90°，所以 $size/2 = f * \pi/2$, $f = size/\pi$, $c = size/2$

2.2 像素投影相机

归一化相平面（深度为1平面）坐标为： $x = \frac{u - c_x}{f_x}, y = \frac{v - c_y}{f_y}, 1$

根据等距假设，有 $\theta = r/f = \sqrt{x^2 + y^2}$, $\varphi = \arctan(y, x)$

这里恢复的是一个方向向量，R=1的单位球面上对应的方向向量（pt_cam）为

$$X = \sin \theta \cos \phi, Y = \sin \theta \sin \phi, Z = \cos \theta$$

- 这里的关键在于，针孔的归一化再乘depth的逻辑是认为相同Z（深度）投影到同一点，但是等距鱼眼假设里是相同角度投影到同一点，也就是成像的是方向向量本身

2.3 相机投影像素

方向向量归一化至单位相平面 $x = X/Z, y = Y/Z$

该点距离z轴投影距离为 $r = \sqrt{x^2 + y^2}$

由于此处为Z=1平面，所以对应的角度 $\theta = \arctan(r)$

根据鱼眼畸变模型，等 θ 处畸变相等，所以畸变后成像点的夹角变为

$$\theta_d = \theta(1 + k_1\theta^2 + k_2\theta^4 + k_3\theta^6 + k_4\theta^8)$$

在等距假设中，由于 θ 和 r 线性关系，所以坐标也等倍放缩：

$$x' = (r'/r)x = (f\theta_d/r)x = (\theta_d/r)x$$

$$y' = (r'/r)y = (f\theta_d/r)y = (\theta_d/r)y$$

转到像素平面上有

$$u = f_x * x' + c_x$$

$$v = f_y * y' + c_y$$

3. 球面展开模型

- 使用经纬度 θ 和 ϕ ， θ 为经度向前为 0° ，向右为正； ϕ 为纬度， 0° 为赤道，上下各 90° ，来描述空间点，并且直接将角度投影到xy上，所以这是一个全向投影，能够描述整个3D空间，同时由于经纬度几乎均匀分布，所以边缘不会再有严重收缩

3.1 像素投影相机

归一化相平面（深度为1平面）坐标为： $\theta = x = \frac{u - c_x}{f_x}, \varphi = y = \frac{v - c_y}{f_y}$

相机坐标系下的方向向量为： $X = \sin \theta \cos \phi, Y = \sin \theta \sin \phi, Z = \cos \theta$

3.2 相机投影像素

$$\theta = \arctan(X/Z)$$

$$\phi = \arctan(Y/\sqrt{X^2 + Z^2})$$

$$u = f\theta + c_x$$

$$v = f\phi + c_y$$

4. 等距柱面展开模型

- 将3D世界投影到一个固定距离的柱面上再成像，一般会定义 $R=f$

设柱面三维真实距离为 R ，像素距离为 r 原始图像真实焦距 F ，像素焦距 f

4.1 虚拟内参

不需要额外处理，用新的最大最小的fov投影一下焦距

```
std::vector<cv::Point2f> points_in = {
    cv::Point2f(0., height_ / 2),
    cv::Point2f(width_, height_ / 2),
    cv::Point2f(width_ / 2, 0.),
    cv::Point2f(width_ / 2, height_),
};
std::vector<cv::Point2f> points_out;
if (raw_d.rows == 4) {
    cv::fisheye::undistortPoints(points_in, points_out, raw_k, raw_d,
                                cv::noArray(), cv::noArray());
} else {
    cv::undistortPoints(points_in, points_out, raw_k, raw_d, cv::noArray(),
                        cv::noArray());
}
float foww = std::abs(std::atan2(points_out[0].x, 1)) +
             std::abs(std::atan2(points_out[1].x, 1));
float fowh = std::abs(std::atan2(points_out[2].y, 1)) +
             std::abs(std::atan2(points_out[3].y, 1));
inner_K_ << new_width / foww, 0., new_width / 2, 0., new_height / fowh,
           new_height / 2, 0., 0., 1.0;
inner_K_inv_ = inner_K_.inverse();
inner_K_mat_ =
    (cv::Mat_<float>(3, 3) << inner_K_(0, 0), inner_K_(0, 1),
     inner_K_(0, 2), inner_K_(1, 0), inner_K_(1, 1), inner_K_(1, 2),
     inner_K_(2, 0), inner_K_(2, 1), inner_K_(2, 2));
scale = 1.;
offset_x = offset_y = 0;
} else {
```

4.2 像素投影相机

- 三维空间的柱面坐标表达为 (θ, h, R) ，柱面转换后有为 $\theta = \frac{u_{cyl} - c_{cylx}}{r}h = \frac{v_{cyl} - c_{cyl y}}{r}$
- 则对应的柱面坐标系坐标系三维方向为（这是标准柱面展开）

$$X = R \sin(\theta) \quad Y = h \quad Z = R \cos(\theta)$$

柱面恢复深度的方法是 $P_{cam} = D/R * P_{cylinder}$

- 某个角度坐标平面

$$x_{img} = f * \theta$$

$$y_{img} = f * \phi$$

$$X = f * \sin(\theta)$$

$$Y = f * \tan(\phi) \text{ 或者 } f * \phi \text{ (视实现而定)}$$

$$Z = f * \cos(\theta)$$

4.3 相机投影像素(这是mono3d的表示)

方向向量归一化至单位相平面 $x = X/Z, y = Y/Z$

利用真空方式加畸变

$$\theta = \arctan(X)$$

$$u = f\theta + c_x$$

$$v = fy + c_y$$

实际上 由于投影到R柱面用的不是y而是h，所以应该是

已知 3D 点 (X, Y, Z) ，要将其投影到柱面图像上。

1. 先转为柱面坐标：

$$\theta = \arctan 2(X, Z), \quad h = \frac{Y}{\sqrt{X^2 + Z^2}}$$

注：如果柱面半径固定为 $R = f$ ，此处可以近似 $h = Y/R$ 。

2. 再映射到像素坐标（等距假设）：

$$u = f_x\theta + c_x, \quad v = f_yh + c_y$$

P(X Y Z) -> 虚拟外参

P(X_CAM, Y_CAM, Z_CAM)

P(x_cam, y_cam, 1)

-> 虚拟内参

$$\theta = \arctan(X)$$

$$u = f\theta + c_x$$

->v

$$f * y_{cam} / \sqrt{x_{cam}^2 + 1} + c_y$$

P->P_cam

$$p_{cam}/Z$$